



O Que o SGBD Garante

Aula 12 · Bloco 3 — O SGBD na Prática

Tudo ou nada, e mais três letras

Nesta aula

1. Duas operações que precisam acontecer **juntas**
2. **COMMIT** e **ROLLBACK**
3. **ACID** em linguagem direta
4. Duas pessoas **ao mesmo tempo**
5. Usuários e **permissões**
6. **Backup** e restauração

Os comandos aqui são gesto, não sintaxe

`INSERT` e `UPDATE` aparecem
nesta aula para serem repetidos,
não aprendidos.

Eles são ensinados na Aula 14.

Aqui interessa o que está **em volta** deles.

Duas operações que precisam acontecer juntas

1. gravar a linha em EMPRESTIMO
2. mudar a situação do exemplar para 'emprestado'

Entre uma e outra existe um instante. Se a energia cair ali, o banco diz "disponível" e o livro está na mochila de alguém.

```
BEGIN;  
    INSERT INTO emprestimo_simples (...) VALUES (...);  
    UPDATE livro SET situacao = 'emprestado' WHERE tombo = 4420;  
COMMIT;
```

COMMIT e ROLLBACK

Comando

O que faz

COMMIT

confirma — tudo passa a valer, de uma vez, para todos

ROLLBACK

desfaz — é como se a transação nunca tivesse começado

Entre o **BEGIN** e o **COMMIT**, **nada disso existe para as outras pessoas**.

E o **ROLLBACK** não precisa ser seu: o banco o executa sozinho quando algo dá errado.

O banco desfazendo por conta própria

```
BEGIN;  
    UPDATE livro SET situacao = 'manutencao' WHERE tombo = 4417; -- correto  
    UPDATE livro SET situacao = 'perdido'     WHERE tombo = 4418; -- viola o CHECK  
COMMIT;
```

```
ERROR: new row violates check constraint "livro_situacao_check"  
ROLLBACK
```

Repare no exemplar 4417

O primeiro comando estava
perfeitamente correto —
e mesmo assim foi desfeito,
porque o segundo falhou.

É esse o contrato: a transação é indivisível
inclusive no fracasso.

ACID, em linguagem direta

 **Atomicidade** — tudo ou nada;

 **Consistência** — de um estado válido a outro estado válido;

 **Isolamento** — transações simultâneas não veem o meio do trabalho uma da outra;

 **Durabilidade** — depois do **COMMIT**, está gravado. Faltou luz? Está lá.

A é sobre falhar no meio · **C** é sobre as suas regras · **I** é sobre os outros · **D** é sobre o disco.

Duas pessoas ao mesmo tempo

Atendente A

Atendente B

lê situação do 4417: disponível

lê situação do 4417: disponível

grava: emprestado para Ana

grava: emprestado para Bruno

O **isolamento** impede isso — e custa: enquanto A escreve, B espera.

Por isso existem **níveis**: você escolhe quanta garantia quer pagar.

⚠️ O padrão não protege de tudo

`READ COMMITTED` protege
de **ler lixo**: ninguém enxerga
transação não confirmada de ninguém.

Não protege de dois processos
decidirem a mesma coisa
a partir da **mesma leitura**.

Você vai ver isso acontecer no desafio 🌶️.

Usuários e permissões

```
CREATE ROLE consulta_biblioteca LOGIN PASSWORD 'trocar_depois';  
  
GRANT CONNECT ON DATABASE curso_bd TO consulta_biblioteca;  
GRANT USAGE ON SCHEMA public TO consulta_biblioteca;  
GRANT SELECT ON livro, emprestimo_simples TO consulta_biblioteca;
```

Essa identidade **lê e não escreve**. Um **INSERT** dela recebe **permission denied**.

É assim que o **nível externo** da Aula 10 vale de verdade.

Conceda o mínimo necessário

E conceda **por papel**, não por pessoa.

A aplicação que só mostra relatório
não precisa de permissão de escrita —

e no dia em que ela tiver uma falha de segurança,
essa decisão é a diferença
entre um vazamento e um desastre.

Backup e restauração

```
pg_dump curso_bd > curso_bd_2026-03-15.sql      # gera  
psql -d curso_bd_novo -f curso_bd_2026-03-15.sql # restaura
```

Transação protege de falha **no meio da operação**. Não protege de alguém apagar a tabela errada, nem do disco morrer.

 **Backup que nunca foi restaurado não é backup — é esperança.**

Exercícios da aula

Na pasta `aula-12/` :

1. `ex01.md` — rode o script e explique o que sobrou do exemplar 4417;
2. `ex02.md` — qual letra do ACID falhou em cada um dos quatro casos?
3. `ex03.md` — crie a identidade `auditoria` e cole o `permission denied` ;
4. `ex04.md` — faça o backup, restaure num banco novo e **prove** que bateu;
5. **Desafio** 🌶️ `ex05.md` — reproduza a atualização perdida em **dois terminais**.

Próxima aula

Aula 13 — SQL DDL

O esquema que você tem em papel
desde a Aula 08
vira um arquivo que qualquer
PostgreSQL do mundo executa.